Model Predictive Control
Prof. M. Zeilinger

Spring 2025

Programming Exercise
Climate Control

Preliminaries

MATLAB

For this project, you need to have MATLAB **R2022b** installed. Since only Matlab 2024b or newer are available in the ETH IT shop, you need to create a Mathworks account with your ETHZ mail address and download MATLAB from the official homepage[1]. Furthermore, the Control Systems Toolbox, the Optimization Toolbox, Simulink and **no other toolboxes** ought to be be installed.

Installation of MPT & Yalmip

We rely on the Multi-Parametric Toolbox (MPT) for set computations and Yalmip[2] to setup MPC controllers in Matlab.

To install MPT, complete the following instructions:

1. Go to `https://www.mpt3.org/` and click on "Installation & updating instructions".
2. Download the file `install_mpt3.m`.
3. Run `install_mpt3.m` in MATLAB.

MPT automatically installs Yalmip.

Provided Files

The provided files of this project are structured in three subdirectories: `templates/`, `testing/` and `utilities/`. Make sure to add the whole provided folder including all files and subdirectories to the **MATLAB path**.

The subdirectory `templates/` contains template files that serve as a basis for your implementation. Do **not** change the input-output structure of the provided functions when implementing your solution. A complete list of deliverables and their input-output signature can be found in Table 3. More information about the functions can be found in the templates and their respective task description.

The folder `testing/` contains a number of encrypted MATLAB `p`-files to test your solution, as well as the `run_tests` function. Usage of the testing framework is described in more detail in the following.

Below you find some important information regarding the provided template and utility functions:

- `templates/generate_params_cc.m` (Function) — This function returns a struct (called `params` in the following) containing parameters that are shared across most of the functions you implement as part of this project. The fields of the `params` struct and their corresponding parameters are detailed in Table 1.

[1] `https://ch.mathworks.com/downloads/`
[2] `https://yalmip.github.io/`

- `utilities/plot_trajectory_cc.m` (Function) — This function takes as input arguments trajectory data in terms of the states and inputs, X_{sim} and U_{sim} as defined in eq. (14), feasibility information `ctrl_info` of the controller, see Task 7, and the `params` struct returned by `generate_params_cc` to create a basic plot. Feel free to modify it as you need.
- `utilities/traj_abs2delta.m` (Function) — This function can be used to transform trajectory data X_{sim}, U_{sim} into delta-coordinates by shifting them according to the steady-state $\boldsymbol{x}_s, \boldsymbol{u}_s$.
- `utilities/traj_delta2abs.m` (Function) — This function can be used to transform trajectory data X_{sim}, U_{sim} into absolute coordinates by shifting them according to the steady-state $\boldsymbol{x}_s, \boldsymbol{u}_s$.

Field	Value
`model.a1o`	$\alpha_{1,\text{o}}$
`model.a2o`	$\alpha_{2,\text{o}}$
`model.a3o`	$\alpha_{3,\text{o}}$
`model.a12`	$\alpha_{1,2}$
`model.a23`	$\alpha_{2,3}$
`model.m1`	m_1
`model.m2`	m_2
`model.m3`	m_3
`model.nx`	n_x
`model.nu`	n_u
`model.nd`	n_d
`model.ny`	n_y
`model.A`	A
`model.B`	B
`model.Bd`	B_d
`model.C`	C
`model.Cd`	C_d
`model.C_ref`	C_{ref}
`model.D`	D
`model.TimeStep`	Δt

Field	Value
`constraints.InputMatrix`	H_u
`constraints.InputRHS`	$\boldsymbol{h}_u$
`constraints.P1Max`	$p_{1,\max}$
`constraints.P1Min`	$p_{1,\min}$
`constraints.P2Max`	$p_{2,\max}$
`constraints.P2Min`	$p_{2,\min}$
`constraints.StateMatrix`	H_x
`constraints.StateRHS`	$\boldsymbol{h}_x$
`constraints.T1Max`	$T_{1,\max}$
`constraints.T1Max`	$T_{1,\min}$
`constraints.T2Max`	$T_{2,\max}$
`constraints.T2Min`	$T_{2,\min}$
`constraints.T3Max`	$T_{3,\max}$
`constraints.T3Min`	$T_{3,\min}$
`exercise.InitialConditionA`	$\boldsymbol{x}_0^A$
`exercise.InitialConditionB`	$\boldsymbol{x}_0^B$
`exercise.InitialConditionC`	$\boldsymbol{x}_0^C$
`exercise.MPCHorizon`	N
`exercise.etaA`	$\boldsymbol{\eta}_A$
`exercise.etaB`	$\boldsymbol{\eta}_B$
`exercise.QdiagOpt`	$\boldsymbol{q}^*$
`exercise.RdiagOpt`	$\boldsymbol{r}^*$
`exercise.SimHorizon`	N_{sim}
`exercise.T_ref`	T_{ref}
`exercise.To`	$T_{\text{o,ref}}$
`exercise.To_max`	$T_{\text{o,max}}$
`exercise.To_min`	$T_{\text{o,min}}$
`exercise.eta_max`	$\boldsymbol{\eta}_{\max}$
`exercise.eta_min`	$\boldsymbol{\eta}_{\min}$

Table 1: `params` struct of system parameters returned by the function `generate_params`.

Testing framework

In this programming exercise we make use of a testing framework. The framework serves two main purposes: to provide you with feedback about your solution during development, and to automatically grade your submission.

To facilitate testing of your submission, most deliverables are given in terms of *functions*, whose input-output behavior is specified in the task description and verified by the testing framework.

To run the tests for all functions, run

```
test_struct = run_tests("all");
```

The testing framework is available for all submitted functions and scripts. To run the tests for one specific function called `function_name` (`function_name` is a string, e.g., `"generate_system_cont_cc"`), run

```
test_struct = run_tests(function_name);
```

The output `test_struct` is a MATLAB struct whose fields are described in Table 2. Note that when running "all" tests, the framework first checks that the tests for any prerequisite functions pass. The testing framework may vary all of the inputs of the function to be tested, so it is important to **avoid hard-coding parameters** such as input and state dimensions etc. As an emphasizing remark, always use the parameters available in the `params` struct (generated by `generate_params_cc`) in your implementation and do not recompute or hard-code them.

Field	Value
`Deliverable`	The name of the tested function.
`Info`	Short summary of the diagnosis. Passed tests are labeled `"OK"`. If an error is caught during the execution of your function, it will display `"ERROR: <message>"`. If the output of your function does not match the expected value, it shows `"Failure: <message>"`.
`TestResults`	An array of MATLAB `TestResult` objects, which you can explore for debugging.

Table 2: `test_struct` struct of test results returned by the function `run_tests`.

In case we (or you) find a bug inside the software framework, it might be necessary to update the testing framework while you are working on the project. In this regard, we highly appreciate your feedback and would like to encourage you to report bugs or unexpected behavior of the software at the dedicated forum section "Programming Exercise - Questions" on Moodle. In any case, the version of the testing framework used for grading will not be changed within the two weeks preceding the hand-in date. Afterwards, if the testing framework flags parts of your submission as incorrect and you would like to insist on the correctness of your solution, you may write **max. 2 pages** report documenting your code; that part of your submission will then be checked manually.

What you have to hand in

Up to three students are allowed to work together on the programming exercise. They will all receive the same grade. As a group, you must read and understand the ETH plagiarism policy here: `http://www.plagiarism.ethz.ch/` - each submitted work will be tested for plagiarism. You must download and fill out the Declaration of Originality, available at the same link[3]. You are not allowed to make this project description, template, or your solution publicly available.

Hand in a single zip-file, where the filename contains your ETH ID numbers ("xx-xxx-xx", "yy-yyyy-yy", "zz-zzzz-zz" including dashes) of all team-members according to this template (note that there are no spaces in the filename):

`xx-xxx-xx_yy-yyyy-yy_zz-zzz-zz.zip`.

The zip-file must contain the following files according to the exercises:

[3] `https://ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/declaration-originality.pdf`

- Files corresponding to the deliverables summarized in Table 3.
- A MAT-file which contains the `test_struct` that is returned by the testing framework (see the description of the testing framework above).
- A PDF file of a scan of the Declaration of Originality signed by all team-members.
- If applicable, a PDF file of your **optional** 2-page report (see the description of the testing framework above).

All files should be placed at the highest level of the zip-folder and **no** other files or sub-folders should be included in the zip-folder; in particular, the files `testing/` and `utilities/` should **not** be submitted. The directory structure and naming of the files **must** follow this template:

```
xx-xxxx-xx_yy-yyyy-yy_zz-zzzz.zz.zip
├── Declaration_of_Originality.pdf
├── test_struct.mat
├── Report.pdf (optional)
└── <All template/*.m and template/*.mat files>
```

The zip-file should be uploaded to Moodle in the Programming Exercise assignment area by just one of the members of the group. The deadline for submission is **01.06.2025, 23:59 h**. Late submissions will not be considered.

Simulation and self-study questions

Some of the questions are marked as simulation or self-study questions. While these types of questions are ungraded, they are intended to guide your learning experience and test your broader understanding beyond the pure implementation of the methods. Simulation questions encourage you to test the developed methods in a setting where the differences between different control methods become apparent. Self-study questions are of a more theoretic and experimental nature; they serve as additional material to prepare you for the exam. We thus strongly recommend to answer and discuss these questions in your group.

Deliverable summary

Task	Function	Inputs	Outputs	Pt.
1	`generate_system_cont_cc`	`params`	A^c, B^c, B_d^c	2
2	`discretize_system_dist`	A^c, B^c, B_d^c, `params`	A, B, B_d	2
3	`generate_constraints_cc`	`params`	H_u, $\boldsymbol{h}_u$, H_x, $\boldsymbol{h}_x$	2
4	`generate_params_cc` (modify)		`params`	1
5	`compute_steady_state`	`params`, $\boldsymbol{d}$	$\boldsymbol{u}_s$, $\boldsymbol{x}_s$	2
6	`generate_params_delta_cc`	`params`	`params_delta`	2
7	`LQR`	Q, R, `params`	`ctrl` (`LQR` object)	2
7	`LQR/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	0
8	`simulate`	$\boldsymbol{x}(0)$, `ctrl`, `params`	X_{sim}, U_{sim}, `ctrl_info`	2
9	`traj_cost`	X_{sim}, U_{sim}, Q, R	$J_{N_{\text{sim}}}$	1
10	`traj_constraints_cc`	X_{sim}, U_{sim}, `params`	$T^{\text{sim}}_{1,\max}$, $T^{\text{sim}}_{2,\min}$, $T^{\text{sim}}_{2,\max}$, $p^{\text{sim}}_{1,\min}$, $p^{\text{sim}}_{1,\max}$, $p^{\text{sim}}_{2,\min}$, $p^{\text{sim}}_{2,\max}$, J_u, `cstr_viol`	3
11	`lqr_tuning_script_cc`	-	`lqr_tuning_script_cc.mat`	2
12	`lqr_maxPI`	Q, R, `params`	$\boldsymbol{H}_{\text{LQR}}$, $\boldsymbol{h}_{\text{LQR}}$	3
14	`MPC`	Q, R, N, `params`	`ctrl` (`MPC` object)	5
14	`MPC/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	0
16	`MPC_TE`	Q, R, N, `params`	`ctrl` (`MPC_TE` object)	2
16	`MPC_TE/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	0
18	`MPC_TS`	Q, R, N, H, $\boldsymbol{h}$, `params`	`ctrl` (`MPC_TS` object)	2
18	`MPC_TS/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	0
21	`MPC_TS_SC_ET`	Q, R, N, H, $\boldsymbol{h}$, S, v, S_{t}, v_{t}, `params`	`ctrl` (`MPC_TS_SC_ET` object)	4
21	`MPC_TS_SC_ET/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	0
22	`MPC_TS_SC_ET_script_cc`	-	`MPC_TS_SC_ET_script_cc.mat`	2
24	`generate_params_robust_cc`	`params`	`params_robust`	2
25	`generate_disturbances_cc`	`params_robust`	W_{sim}	1
26	`simulate_uncertain`	$\boldsymbol{x}_0$, `ctrl`, W_{sim}, `params_robust`	X_{sim}, U_{sim}, `ctrl_info`	1
27	`check_RPI`	H_{tube}, h_{tube}, H_w, h_w, K_{tube}, `params_robust`	`is_rpi`	1
29	`compute_tightening`	K_{tube}, H_{tube}, $\boldsymbol{h}_{\text{tube}}$, `params_robust`	`params_robust_tube`	3
30	`MPC_TUBE`	Q, R, N, H_N, $\boldsymbol{h}_N$, H_{tube}, $\boldsymbol{h}_{\text{tube}}$, K_{tube}, `params_robust_tube`	`ctrl` (`MPC_TUBE` object)	4
30	`MPC_TUBE/eval`	$\boldsymbol{x}$	$\boldsymbol{u}$, `ctrl_info`	1
31	`MPC_TUBE_script_cc`	-	`MPC_TUBE_params_cc.mat`	2

Table 3: Deliverable summary

System description

In this project you will implement an MPC controller for the temperature regulation of a delivery truck with three temperature zones, two of which are equipped with cooling units to regulate the desired temperatures. The truck is illustrated in Figure 1 and consists of the following zones.

Zone 1 Deep freeze temperature zone at $-21°C$ for frozen goods actuated by *Cooling Unit 1*. In order to ensure food safety, the temperature may not exceed $-15°C$ at any time.

Zone 2 Cooled temperature zone at $0.3°C$ for fresh goods actuated by *Cooling Unit 2*. The temperature shall never exceed $4°C$. Additionally, freezing temperatures below $0°C$ must be avoided at all times.

Zone 3 Unregulated temperature zone for non-perishable goods.

Each zone is equipped with a sensor providing the current temperature of the respective zone. The system is actuated by cooling units in Zone 1 and 2 with different capacities.

Cooling Unit 1 Provides cooling power between $-2500\,\mathrm{W}$ and $0\,\mathrm{W}$.

Cooling Unit 2 Provides cooling power between $-2000\,\mathrm{W}$ and $0\,\mathrm{W}$.

Your task is to design a temperature controller for the delivery truck, which tracks the desired temperature and satisfies the food safety constraints at all times.

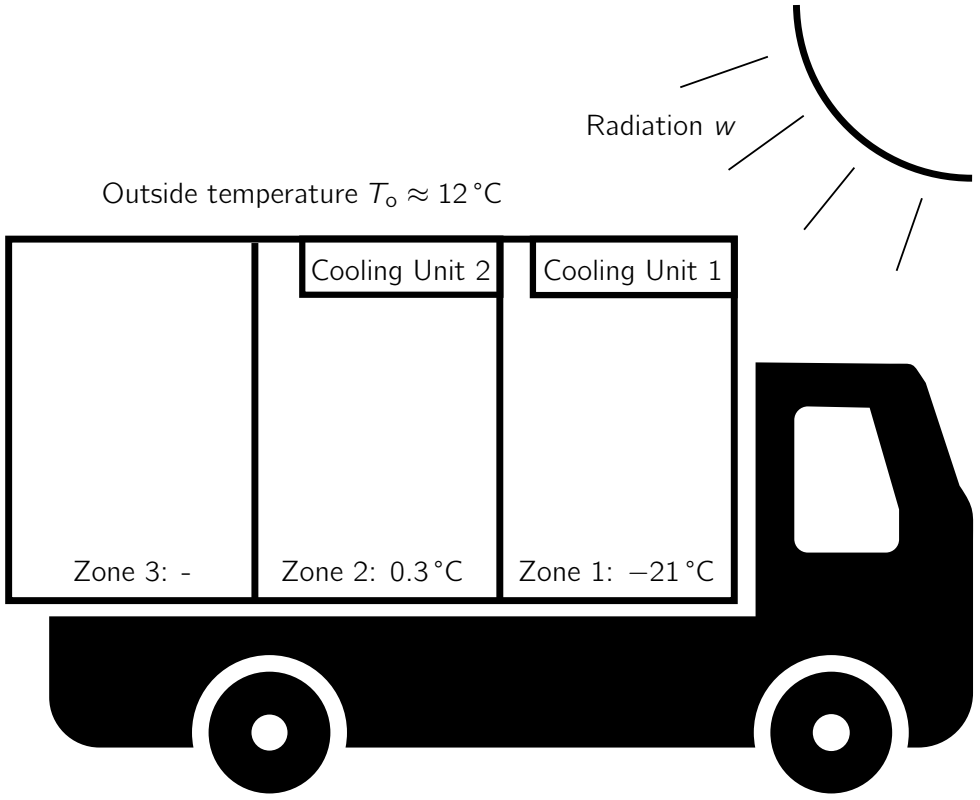

Figure 1: Delivery truck with different climate zones. Zones 1 and 2 are equipped with an integrated cooling unit regulating the respective temperatures.

Modeling

We model the system using simplified heat flows, consisting of flows between the zones, as well as from the outside and additional heat sources, e.g., by radiation. The outside temperature T_o and external heat flux $\boldsymbol{\eta} = \begin{bmatrix}\eta_1, \eta_2, \eta_3\end{bmatrix}^\top$ are, for now, considered to be constant such that the system dynamics can be modelled using the following differential equations

$$\begin{aligned} m_1 \dot{T}_1^c(t) &= \alpha_{1,2}(T_2^c(t) - T_1^c(t)) + \alpha_{1,o}(T_o - T_1^c(t)) + p_1^c(t) + \eta_1 \\ m_2 \dot{T}_2^c(t) &= \alpha_{1,2}(T_1^c(t) - T_2^c(t)) + \alpha_{2,3}(T_3^c(t) - T_2^c(t)) + \alpha_{2,o}(T_o - T_2^c(t)) + p_2^c(t) + \eta_2 \\ m_3 \dot{T}_3^c(t) &= \alpha_{2,3}(T_2^c(t) - T_3^c(t)) + \alpha_{3,o}(T_o - T_3^c(t)) + \eta_3, \end{aligned} \tag{1}$$

with initial condition $\boldsymbol{x}_0 = [T_1^c(0), T_2^c(0), T_3^c(0)]^\top$, where $T_i^c(t)$ is the temperature of zone i at time $t \geq 0$, $p_i(t)$ is the cooling power of unit i, m_i is its thermal mass, d_i an external heat flux and $\alpha_{i,j}$ the thermal conductivity between zone i and j, or the outside (o).

Your first task is to bring the system in standard form for regulation to a steady-state. Therefore, denote by

$$\boldsymbol{x}(t) := \begin{bmatrix} T_1^c(t) \\ T_2^c(t) \\ T_3^c(t) \end{bmatrix} \in \mathbb{R}^{n_x}, \qquad \boldsymbol{u}(t) := \begin{bmatrix} p_1^c(t) \\ p_2^c(t) \end{bmatrix} \in \mathbb{R}^{n_u}, \qquad \boldsymbol{d} := \begin{bmatrix} \alpha_{1,o} \\ \alpha_{2,o} \\ \alpha_{3,o} \end{bmatrix} T_o + \boldsymbol{\eta} \in \mathbb{R}^{n_d}, \tag{2}$$

the system state, control input and disturbance, respectively.

Deliverables

1. Rewrite ODE (1) in the following form:

$$\dot{\boldsymbol{x}}(t) = A^c \boldsymbol{x}(t) + B^c \boldsymbol{u}(t) + B_d^c \boldsymbol{d}, \tag{3}$$

   where $B_d^c = \text{diag}([\frac{1}{m_1}, \frac{1}{m_2}, \frac{1}{m_3}])$ is the disturbance input matrix. Implement a function called `generate_system_cont_cc` that takes the `params` struct as input and returns the matrices A^c, B^c, B_d^c. *2 pt.*

2. Discretize the continuous-time ODE with sampling time $\Delta t = 60$ s (defined in the params struct as field `params.model.TimeStep`) such that the resulting discrete-time dynamics are given by the difference equation

$$\boldsymbol{x}(k+1) = A\boldsymbol{x}(k) + B\boldsymbol{u}(k) + B_d \boldsymbol{d}, \tag{4}$$

   where $\boldsymbol{x}(k) := \boldsymbol{x}(k\Delta t)$, $\boldsymbol{u}(k) := \boldsymbol{u}(k\Delta t)$ and $\boldsymbol{d} := \boldsymbol{d}$ for $\boldsymbol{x}(0) = \boldsymbol{x}(0)$. Assume piece-wise constant input signals within sampling intervals, i.e., $\boldsymbol{u}(k\Delta t + \tau) = \boldsymbol{u}_k = \text{const.}$ for all $\tau \in [0, \Delta t)$. Implement your solution as a function `discretize_system_dist`, that computes and returns A, B, B_d based on the continuous-time matrices A^c, B^c, B_d^c and the `params` struct as inputs.
   Hint: You can use the MATLAB function c2d. *2 pt.*

3. Find matrices $H_x \in \mathbb{R}^{6 \times n_x}$, $H_u \in \mathbb{R}^{4 \times n_u}$, as well as vectors $\boldsymbol{h}_x \in \mathbb{R}^6$, $\boldsymbol{h}_u \in \mathbb{R}^4$ such that

$$H_x \boldsymbol{x}_d(k) \leq \boldsymbol{h}_x, \tag{5}$$
$$H_u \boldsymbol{u}_d(k) \leq \boldsymbol{h}_u, \tag{6}$$

   encode box constraints $x_{i,\min} \leq x_i \leq x_{i,\max}$, $u_{j,\min} \leq u_j \leq u_{j,\max}$, for all state and input components $i = 1, \ldots, n_x$, $j = 1, \ldots, n_u$, respectively. For unconstrained directions of the state constraints, set the maximum and minimum values to $+1000°\text{C}$ and $-1000°\text{C}$, respectively. Implement a function `generate_constraints_cc`, that takes as input the parameter struct `params` and outputs $H_u, \boldsymbol{h}_u, H_x, \boldsymbol{h}_x$. *2 pt.*

4. Modify the function `generate_params_cc` to also compute A, B, B_d, H_u, $\boldsymbol{h}_u$, H_x, $\boldsymbol{h}_x$ as part of its execution (using the functions you implemented above) and add the corresponding fields as defined in Table 1 to the output `struct`. The output of the function `generate_params_cc` should now exactly give a `struct` as specified in Table 1.
   Important: When using $H_u, \boldsymbol{h}_u, H_x, \boldsymbol{h}_x$ in the following exercise, avoid calling the function `generate_constraints_cc` to re-generate them; instead, always use the values saved in the `params` struct. 1 *pt.*

5. Let a measurement vector $\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{d}) \in \mathbb{R}^{n_y}$ be defined by

$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{d}) := C\boldsymbol{x} + C_d\boldsymbol{d}, \tag{7}$$

   where $C \in \mathbb{R}^{n_y \times n_x}$, $C_d \in \mathbb{R}^{n_y \times n_d}$, are defined in the `params` struct. Implement a function `compute_steady_state` that takes as input the parameter struct `params` (with the temperature reference $\boldsymbol{T}_{\text{ref}} \in \mathbb{R}^2$ for states 1 and 2 given in `params.exercise.T_ref`) and a disturbance estimate $\boldsymbol{d}$, and outputs the corresponding steady state $\boldsymbol{x}_s$ and steady-state input $\boldsymbol{u}_s$ of (4), such that offset-free tracking may be achieved, i.e., $C_{\text{ref}}\boldsymbol{y}(\boldsymbol{x}_s, \boldsymbol{d}) = \boldsymbol{T}_{\text{ref}}$. 2 *pt.*

Given the steady-state of the system, the system dynamics can be expressed in the form of a discrete-time "delta-formulation" model:

$$\Delta\boldsymbol{x}_d(k+1) = A\Delta\boldsymbol{x}_d(k) + B\Delta\boldsymbol{u}_d(k), \tag{8}$$

which can be used for regulating the system to the computed steady-state.

6. Implement a function `generate_params_delta_cc` that takes as input the `params` struct, based on which it computes a disturbance

$$\boldsymbol{d} = \begin{bmatrix} \alpha_{1,\text{o}} \\ \alpha_{2,\text{o}} \\ \alpha_{3,\text{o}} \end{bmatrix} T_{\text{o,ref}} + \boldsymbol{\eta}_A \tag{9}$$

   and the corresponding steady-state $\boldsymbol{x}_s, \boldsymbol{u}_s$. Based on these values, it should adjust and *overwrite the constraints* (5),(6) *and initial conditions* $\boldsymbol{x}_0^A, \boldsymbol{x}_0^B, \boldsymbol{x}_0^C$ of the system, such that they are equivalently expressed in the delta-coordinates

$$\Delta\boldsymbol{x}_d(k) = \boldsymbol{x}_d(k) - \boldsymbol{x}_s, \tag{10}$$
$$\Delta\boldsymbol{u}_d(k) = \boldsymbol{u}_d(k) - \boldsymbol{u}_s, \tag{11}$$

   as well as add fields for the steady state $\boldsymbol{u}_s$, $\boldsymbol{x}_s$. The corresponding fields in the `params_delta` struct can be found in Table 4. The function should return the modified struct `params_delta`.

   2 *pt.*

Unconstrained Optimal Control

Notation: For brevity, and with a slight abuse of notation, we drop the subscript in the following and write $x(k)$ and $u(k)$ to refer to the discrete-time state and input variables, $x_d(k)$ and $u_d(k)$, respectively. Additionally, in the following we consider the regulation to the computed steady-state and omit Δ, referring to, e.g., $\Delta x(k)$ by writing $x(k)$. Analogously, we denote by `params` the struct `params_delta` in delta-coordinates, i.e. **if not required differently by your implementation, use** `params_delta` **in the place of** `params` **from here on**.

Field	Value
`exercise.InitialConditionA`	$\Delta \boldsymbol{x}_0^A$
`exercise.InitialConditionB`	$\Delta \boldsymbol{x}_0^B$
`exercise.InitialConditionC`	$\Delta \boldsymbol{x}_0^C$
`constraints.InputMatrix`	ΔH_u
`constraints.InputRHS`	$\Delta \boldsymbol{h}_u$
`constraints.StateMatrix`	ΔH_x
`constraints.StateRHS`	$\Delta \boldsymbol{h}_x$
`exercise.u_s`	$\boldsymbol{u}_s$
`exercise.x_s`	$\boldsymbol{x}_s$

Table 4: Added/modified fields in the `params_delta` struct (other fields same as `params`).

The aim of the following tasks is to design a discrete-time infinite-horizon linear quadratic regulator (LQR) that satisfies the mission requirements. The infinite-horizon LQR controller

$$\boldsymbol{u}(k) := F_\infty \boldsymbol{x}(k) \tag{12}$$

is defined such that it minimizes the infinite-horizon quadratic cost

$$J_\infty(\boldsymbol{x}(0)) := \sum_{k=0}^{\infty} \boldsymbol{x}(k)^\top Q \boldsymbol{x}(k) + \boldsymbol{u}(k)^\top R \boldsymbol{u}(k), \tag{13}$$

for some positive definite weighting matrices $Q \in \mathbb{R}^{n_x \times n_x}$ and $R \in \mathbb{R}^{n_u \times n_u}$. As the control requirements and constraints cannot be encoded directly, but rather have to follow from the choice of the weights, the main difficulty is to find Q and R that lead to a satisfactory system response. Therefore a parameter study is to be conducted. To simplify the parameter study, we define $\boldsymbol{q} \in \mathbb{R}^{n_x \times 1}$, $\boldsymbol{q} := \begin{bmatrix} q_1 & q_2 & q_3 \end{bmatrix}^\top$ as well as $\boldsymbol{r} \in \mathbb{R}^{n_u \times 1}$, $\boldsymbol{r} := \begin{bmatrix} r_1 & r_2 \end{bmatrix}^\top$ and choose the parametrization $Q := \mathrm{diag}(\boldsymbol{q})$ and $R = \mathrm{diag}(\boldsymbol{r})$.

Tasks

7. Design an infinite-horizon LQR controller for given inputs Q and R. Implement your solution as a *class* in the template file `LQR.m`. Note that the class template has two functions, the constructor `LQR(Q,R)` and `eval(x)`. For numerical efficiency, the feedback matrix F_∞ should only be computed at initialization once (in the constructor of the class) and stored as a class property `K` $:= L_\infty$. The class property can then be accessed at every call to the `eval` function, which computes the feedback of the controller according to equation (12) without additional computational overhead. Note that, in addition to the control action $\boldsymbol{u}$, the `eval` function also returns a struct `ctrl_info` containing additional information about the control output. For the LQR controller, the `ctrl_info` struct has only one field, `ctrl_feas`, indicating the feasibility of the control problem; it suffices to always set

   `ctrl_info.ctrl_feas` $=$ `true`.

   For convenience, we have already added the `eval` function to the template `LQR.m`. More details can be found in the template file. 2 *pt.*

8. Simulate the closed-loop system for a given initial condition $\boldsymbol{x}(0)$, controller object `ctrl` and number of time steps N_{sim}. Implement your solution in the function `simulate`, which takes $\boldsymbol{x}(0)$, `ctrl`, `params` as inputs and outputs the closed-loop trajectory according to equation (8) in terms of the matrices $X_{\mathrm{sim}} \in \mathbb{R}^{n_x \times (N_{\mathrm{sim}}+1)}$, $U_{\mathrm{sim}} \in \mathbb{R}^{n_u \times N_{\mathrm{sim}}}$, with

$$X_{\mathrm{sim}} := \begin{bmatrix} \boldsymbol{x}(0) & \dots & \boldsymbol{x}(N_{\mathrm{sim}}) \end{bmatrix}, \qquad U_{\mathrm{sim}} := \begin{bmatrix} \boldsymbol{u}(0) & \dots & \boldsymbol{u}(N_{\mathrm{sim}}-1) \end{bmatrix}, \tag{14}$$

as well as an N_{sim}-dimensional array of the `ctrl_info` structs described in Task 7. Note that the simulation should explicitly **not** include saturation of the inputs, i.e., compute the evolution of the linear system according to the raw input of the feedback controller. 2 *pt.*

9. Implement a function `traj_cost` that takes as input arguments $Q \in \mathbb{R}^{n_x \times n_x}$, $R \in \mathbb{R}^{n_u \times n_u}$, as well as trajectory data X_{sim}, U_{sim} as in equation (14) and outputs the closed-loop quadratic cost for a given trajectory, i.e.,

$$J_{N_{\text{sim}}} := \sum_{k=0}^{N_{\text{sim}}-1} \boldsymbol{x}(k)^\top Q \boldsymbol{x}(k) + \boldsymbol{u}(k)^\top R \boldsymbol{u}(k).$$

1 *pt.*

10. Check the satisfaction of the constraints for a given trajectory. Implement a function called `traj_constraints_cc`, that takes as input the trajectory data $X_{\text{sim}} \in \mathbb{R}^{n_x \times N_{\text{sim}}}, U_{\text{sim}} \in \mathbb{R}^{n_u \times N_{\text{sim}}}$ in **absolute** coordinates and computes as outputs

    - the maximum temperature in Zone 1, $T^{\text{sim}}_{1,\max} = \max_{k \in [0, N_{\text{sim}}]} T^c_1(k\Delta T)$,
    - the minimum temperature in Zone 2, $T^{\text{sim}}_{2,\min} = \min_{k \in [0, N_{\text{sim}}]} T^c_2(k\Delta T)$,
    - the maximum temperature in Zone 2, $T^{\text{sim}}_{2,\max} = \max_{k \in [0, N_{\text{sim}}]} T^c_2(k\Delta T)$,
    - the minimum cooling power in Zone 1, $p^{\text{sim}}_{1,\min} = \min_{k \in [0, N_{\text{sim}}]} p^c_1(k\Delta T)$,
    - the maximum cooling power in Zone 1, $p^{\text{sim}}_{1,\max} = \max_{k \in [0, N_{\text{sim}}]} p^c_1(k\Delta T)$,
    - the minimum cooling power in Zone 2, $p^{\text{sim}}_{2,\min} = \min_{k \in [0, N_{\text{sim}}]} p^c_2(k\Delta T)$,
    - the maximum cooling power in Zone 2, $p^{\text{sim}}_{2,\max} = \max_{k \in [0, N_{\text{sim}}]} p^c_2(k\Delta T)$,
    - the closed-loop finite-horizon input cost, $J_u := \sum_{k=0}^{N_{\text{sim}}-1} \boldsymbol{u}(k)^\top \boldsymbol{u}(k)$,
    - a boolean flag `cstr_viol` indicating if the trajectory contains any constraint violations, i.e., `cstr_viol` = false if and only if all constraints provided in the task description on page 1 are satisfied.

    The function should work with any length of the trajectory data, i.e. it should be able to handle any N_{sim}. 3 *pt.*

11. Design an LQR controller for the system such that for the initial condition $\boldsymbol{x}_0^A$, the following properties hold for the resulting, simulated closed-loop system:

    - Input and state constraints are satisfied for all time steps $k \in [0, 60]$.
    - The closed-loop system approaches the reference reasonably fast, more precisely, it holds that

$$|\Delta \boldsymbol{x}_1(30)| \leq 3 \cdot 10^{-1}, \qquad |\Delta \boldsymbol{x}_1(60)| \leq 3 \cdot 10^{-2}, \tag{15}$$

$$|\Delta \boldsymbol{x}_2(30)| \leq 2 \cdot 10^{-2}, \qquad |\Delta \boldsymbol{x}_2(60)| \leq 2 \cdot 10^{-3}. \tag{16}$$

    Provide a script `lqr_tuning_script_cc.m` that runs a closed-loop simulation based on your tuning of Q and R and saves the following information as a MAT-file `lqr_tuning_script_cc.mat`:

2 *pt.*

Variable	Value
`Q`	Q
`R`	R
`X`	X_{sim} (in delta coordinates)
`U`	U_{sim} (in delta coordinates)

Table 5: Variables contained in `lqr_tuning_script_cc.mat`.

A first model predictive controller

After designing the unconstrained optimal controller, in this section you will design a first simple model predictive controller. In MPC, the control sequence $U := \{\boldsymbol{u}_0, \dots \boldsymbol{u}_{N-1}\}$ is computed as the solution of an optimization problem over a prediction horizon of N steps. Feedback is introduced by only applying the first element of the sequence, $\boldsymbol{u}(0) := \boldsymbol{u}_0$; in the next time step, the optimal input sequence is recomputed based on updated state measurements. Note that we write predicted control inputs with a lower subscript, e.g., $\boldsymbol{u}_0$, to differentiate them from the implemented control inputs $u(0)$. The notation of the states is chosen analogously.

For the following exercises, we use $Q^* := \text{diag}(\boldsymbol{q}^*)$ and $R^* := \text{diag}(\boldsymbol{r}^*)$, with

$$\boldsymbol{q}^* := \begin{bmatrix} q_1^* \\ q_2^* \\ q_3^* \end{bmatrix} := \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{r}^* := \begin{bmatrix} r_1^* \\ r_2^* \end{bmatrix} := \begin{bmatrix} 2.4 \\ 0.5 \end{bmatrix} \cdot 10^{-5}$$

Nevertheless, feel free to also experiment with your best values obtained from the LQR controller tuning in Task 11 and see how it affects the MPC closed-loop performance.

Tasks

12. Explicitly compute the maximum positively invariant set $\mathcal{X}_{LQR}$ under application of the LQR controller. Let $\boldsymbol{x}_{LQR}(k)$ and $\boldsymbol{u}_{LQR}(k)$, $k = 0, \dots, \infty$ be the infinite-horizon closed-loop state and input sequence resulting from application of the LQR controller (12) to system (4), for some initial condition $\boldsymbol{x}_{LQR}(0) = \boldsymbol{x}(0)$. Then, the set $\mathcal{X}_{LQR}$ is defined by

$$\mathcal{X}_{LQR} := \{\boldsymbol{x} \mid \boldsymbol{x}_{LQR}(0) = \boldsymbol{x},\ H_x \boldsymbol{x}_{LQR}(k) \leq \boldsymbol{h}_x,\ H_u \boldsymbol{u}_{LQR}(k) \leq \boldsymbol{h}_u \text{ for all } k \geq 0\}. \tag{17a}$$

    Implement a function `lqr_maxPI`, which takes as inputs Q, R, `params` and returns $H_{\text{LQR}} \in \mathbb{R}^{n_H \times n_x}$ and $\boldsymbol{h}_{\text{LQR}} \in \mathbb{R}^{n_H}$ such that $\mathcal{X}_{LQR} = \{\boldsymbol{x} \mid H_{\text{LQR}} \boldsymbol{x} \leq \boldsymbol{h}_{\text{LQR}}\}$.
    Hint: You can use the MPT toolbox[4] for this task. 3 *pt.*

13. *[Self-study]: Check whether $\boldsymbol{x}_0^A$, $\boldsymbol{x}_0^B$, and $\boldsymbol{x}_0^C$ are contained in $\mathcal{X}_{LQR}$. What can you conclude from the result with respect to state and input constraint satisfaction under the LQR controller for these initial conditions?*

14. Implement a model predictive controller that solves the open-loop optimization problem

$$\min_{U,X} \sum_{i=0}^{N-1} \boldsymbol{x}_i^\top Q \boldsymbol{x}_i + \boldsymbol{u}_i^\top R \boldsymbol{u}_i + l_f(\boldsymbol{x}_N) \tag{18a}$$

$$\text{s.t. } \boldsymbol{x}_0 = \boldsymbol{x}(k) \tag{18b}$$

$$\boldsymbol{x}_{i+1} = A\boldsymbol{x}_i + B\boldsymbol{u}_i, \qquad i = 0, \dots, N-1 \tag{18c}$$

$$H_x \boldsymbol{x}_i \leq \boldsymbol{h}_x, \qquad i = 0, \dots, N \tag{18d}$$

$$H_u \boldsymbol{u}_i \leq \boldsymbol{h}_u, \qquad i = 0, \dots, N-1 \tag{18e}$$

[4] `https://www.mpt3.org/`; in particular, see `https://www.mpt3.org/UI/Filters` for constraint specification.

at each time step, where $l_f(\boldsymbol{x}) = J_\infty(\boldsymbol{x})$ is equal to the LQR infinite-horizon cost (13). Implement the model predictive controller as a class `MPC`, which creates a YALMIP solver object during initialization with Q, R, N, and solves the optimization problem in the `eval` method. Note that in this case, at each call of `eval` the `ctrl_info` struct has two additional fields: `ctrl_info.objective`, which should contain the value of the objective function for the optimizer U^* of (18), and `ctrl_info.solvetime`, which should contain the time required to solve the problem (18). Again, we have provided you with the full `eval` method. More details can be found in the template file.
Hint 1: The LQR infinite-horizon cost is of the form $J_\infty(\boldsymbol{x}) = \boldsymbol{x}^\top P \boldsymbol{x}$, where $P \in \mathbb{R}^{n_x \times n_x}$ is a constant matrix that can be computed during the initialization of the controller.
Hint 2: You can use the MATLAB functions `tic` and `toc` to get an estimate of the required solve time.
Hint 3: Consider the YALMIP documentation for setting up the model predictive controller: `https://yalmip.github.io/example/standardmpc/`. 5 *pt.*

15. *[Simulation]: Simulate the closed-loop system starting from $\boldsymbol{x}_0^A$ with the LQR and the model predictive controller for the same choice of $Q := Q^*$, $R := R^*$ and with $N := 30$. Do the same with $\boldsymbol{x}_0^B$ as initial condition. How do the controllers perform with respect to closed-loop constraint satisfaction and closed-loop cost?*

MPC with theoretical closed-loop guarantees

In this part, the task is to formulate a model predictive controller that provides guaranteed closed-loop state and input constraint satisfaction and renders the origin an asymptotically stable equilibrium point of the closed-loop system.

Tasks

16. Implement a model predictive controller based on the MPC problem

$$\begin{aligned}
\min_{U,X} \quad & \sum_{i=0}^{N-1} \boldsymbol{x}_i^\top Q \boldsymbol{x}_i + \boldsymbol{u}_i^\top R \boldsymbol{u}_i && && \text{(19a)}\\
\text{s.t.} \quad & \boldsymbol{x}_0 = \boldsymbol{x}(k) && && \text{(19b)}\\
& \boldsymbol{x}_{i+1} = A\boldsymbol{x}_i + B\boldsymbol{u}_i, && i = 0, \dots, N-1 && \text{(19c)}\\
& H_x \boldsymbol{x}_i \leq \boldsymbol{h}_x, && i = 0, \dots, N-1 && \text{(19d)}\\
& H_u \boldsymbol{u}_i \leq \boldsymbol{h}_u, && i = 0, \dots, N-1 && \text{(19e)}\\
& \boldsymbol{x}_N = 0, && && \text{(19f)}
\end{aligned}$$

in the class template `MPC_TE`, with the same input arguments as the `MPC` class. 2 *pt.*

17. *[Self-study]: Why is the origin an asymptotically stable equilibrium point for the resulting closed-loop system, given that* (19) *is feasible for $\boldsymbol{x}(0)$?*

18. Implement another model predictive controller based on the MPC problem

$$\min_{U} \sum_{i=0}^{N-1} \boldsymbol{x}_i^\top Q \boldsymbol{x}_i + \boldsymbol{u}_i^\top R \boldsymbol{u}_i + l_\mathrm{f}(\boldsymbol{x}_N) \tag{20a}$$

$$\text{s.t. } \boldsymbol{x}_0 = \boldsymbol{x}(k) \tag{20b}$$

$$\boldsymbol{x}_{i+1} = A\boldsymbol{x}_i + B\boldsymbol{u}_i, \qquad i = 0, \dots, N-1 \tag{20c}$$

$$H_x \boldsymbol{x}_i \leq \boldsymbol{h}_x, \qquad i = 0, \dots, N-1 \tag{20d}$$

$$H_u \boldsymbol{u}_i \leq \boldsymbol{h}_u, \qquad i = 0, \dots, N-1 \tag{20e}$$

$$H\boldsymbol{x}_N \leq \boldsymbol{h}, \tag{20f}$$

in the class template `MPC_TS`, where $l_\mathrm{f}(\boldsymbol{x}) := J_\infty(\boldsymbol{x})$ is the LQR infinite-horizon cost and H, $\boldsymbol{h}$, define a polytopic terminal set. 2 *pt.*

19. *[Self-study]: How can the terminal set be chosen for the given system to ensure that the closed-loop dynamics is stable and satisfies constraints when initialized with an initial condition for which* (20) *is feasible?*

20. *[Simulation]: Simulate the closed-loop system with the three MPCs* (18), (19), (20) *starting from* $\boldsymbol{x}_0^A$ *for the same choice of* $Q := Q^*$, $R := R^*$ *and horizon length* $N = 30$ *and compare them in terms of the feasibility of the open-loop optimization problems, as well as their constraint satisfaction and cost in closed-loop. Test also different horizon lengths* $N = 40 \dots 20$. *What do you observe?*

Soft constraints

In practical implementations, model predictive controllers such as (20) can become infeasible despite the provided theoretical guarantees, for instance due to unmodeled disturbances or model mismatch. This problem can be addressed by using soft constraints, providing a recovery mechanism given that the original problem is infeasible. Your task is to design a soft-constrained model predictive controller, which provides the same control inputs as (20), if (20) is feasible, but may provide a feasible solution even when (20) is infeasible.

Tasks

21. Introduce slack variables $\boldsymbol{\epsilon}_i \in \mathbb{R}^{n_{\mathrm{constr},x}}$, $i = 0, \dots, N$, $\boldsymbol{\epsilon}_\mathrm{t} \in \mathbb{R}^{n_\mathrm{t}}$, into the MPC problem (20) to restore feasibility in the case of state and terminal constraint violations. Implement a model predictive controller based on the MPC problem

$$\min_{U,X,E} \quad \sum_{i=0}^{N-1} \boldsymbol{x}_i^\top Q \boldsymbol{x}_i + \boldsymbol{u}_i^\top R \boldsymbol{u}_i + l_\mathrm{f}(\boldsymbol{x}_N) + \sum_{i=0}^{N-1} \boldsymbol{\epsilon}_i^\top S \boldsymbol{\epsilon}_i + v \|\boldsymbol{\epsilon}_i\|_1 + \boldsymbol{\epsilon}_\mathrm{t}^\top S_\mathrm{t} \boldsymbol{\epsilon}_\mathrm{t} + v_\mathrm{t} \|\boldsymbol{\epsilon}_\mathrm{t}\|_1 \tag{21a}$$

$$\text{s.t. } \boldsymbol{x}_0 = \boldsymbol{x}(k) \tag{21b}$$

$$\boldsymbol{x}_{i+1} = A\boldsymbol{x}_i + B\boldsymbol{u}_i, \quad i = 0, \dots, N-1, \tag{21c}$$

$$H_x \boldsymbol{x}_i \leq \boldsymbol{h}_x + \boldsymbol{\epsilon}_i, \quad i = 0, \dots, N-1, \tag{21d}$$

$$H_u \boldsymbol{u}_i \leq \boldsymbol{h}_u, \quad i = 0, \dots, N-1, \tag{21e}$$

$$H\boldsymbol{x}_N \leq \boldsymbol{h} + \boldsymbol{\epsilon}_\mathrm{t} \tag{21f}$$

$$\boldsymbol{\epsilon}_i \geq 0, \quad i = 0, \dots, N-1, \tag{21g}$$

$$\boldsymbol{\epsilon}_\mathrm{t} \geq 0 \tag{21h}$$

in the class `MPC_TS_SC_ET`. The class shall be initialized with the same parameters as `MPC_TS`, plus the additional values for $S \in \mathbb{R}^{n_{\text{constr},x} \times n_{\text{constr},x}}$, $v \in \mathbb{R}$ and $S_\text{t} \in \mathbb{R}^{n_t \times n_t}$, $v_\text{t} \in \mathbb{R}$ for the constraint violation penalty with $v, v_\text{t} \gg 0$. Thereby $n_{\text{constr},x}$ denotes the number of state constraints as given by the state constraint definition in the `params` struct.
Hint: To avoid numerical errors, it might be necessary to switch to an MPC implementation without *substitution, see the "Implicit prediction form" YALMIP tutorial:* `https://yalmip.github.io/example/standardmpc/`. 4 *pt.*

22. Choose S,v,S_t,v_t such that the controller based on the soft-constrained MPC problem (21) returns the same control inputs as the controller based on the MPC problem (20), whenever (20) is feasible, for the same choice of weighting matrices Q^*, R^* and horizon length $N = 30$. Verify your selection in simulation by providing a script called `MPC_TS_SC_ET_script_cc.m`, which verifies that, with your choice of S,v,S_t,v_t, the soft-constrained controller `MPC_TS_SC_ET` returns the same solution as `MPC_TS` for initial condition $\boldsymbol{x}_0^B$ while returning a solution for initial condition $\boldsymbol{x}_0^C$ (which is infeasible for `MPC_TS`). Provide the script and save your selected values in the MAT-file `MPC_TS_SC_ET_script_cc.mat`, using the following naming convention:

Variable	Value
`v`	v
`S`	S
`v_t`	v_t
`S_t`	S_t

Table 6: Variables contained in `MPC_TS_SC_ET_script_cc.mat`.

2 *pt.*

23. *[Simulation]: Repeat the simulation for the initial condition $\boldsymbol{x}_0^C$ and experiment with different values for S and v in the soft-constrained formulation* (21)*. How does the choice of S and v influence the closed-loop behavior?*

Robust MPC

If we want to ensure satisfaction of constraints even under unmodeled disturbances, we can use a robust MPC approach. The goal of this exercise part is to implement a tube-based MPC in order to robustly guarantee constraint satisfaction in presence of disturbances. The remaining tasks will guide you through the specification of the disturbance bounds, tube MPC implementation and its simulation in the climate controlled truck.
For this part, we consider system (22) with time-varying disturbance as follows:

$$\begin{aligned} \boldsymbol{x}(k+1) &= A\boldsymbol{x}(k) + B\boldsymbol{u}(k) + \boldsymbol{w}(k), \\ \text{with} \quad \boldsymbol{w}(k) = B_d\boldsymbol{d}(k) &= B_d \left(\begin{bmatrix} \alpha_{1,o} \\ \alpha_{2,o} \\ \alpha_{3,o} \end{bmatrix} \cdot (T_o(k) - T_{\text{o,ref}}) + \boldsymbol{\eta}(k) \right) \in \mathbb{R}^{n_x}. \end{aligned} \tag{22}$$

Note that we here subtract the constant disturbance $T_{\text{o,ref}} = \text{const.}$ that we already compensated for when implementing the system matrices in delta formulation in Taks 5 and 6. Further, we assume known bounds on the outside temperature $T_o(k) \in \mathbb{R}$ and the radiation $\boldsymbol{\eta}(k) \in \mathbb{R}^{n_x}$ according to Equation (23).

$$\begin{aligned} T_{o,\min} \leq T_\text{o}(k) \leq T_{o,\max} \\ \eta_{i,\min} \leq \eta_i(k) \leq \eta_{i,\max} \qquad i = 1, \ldots, n_x. \end{aligned} \tag{23}$$

The specific values of the bounds (23) are given in the function `generate_params_cc`, according to Table 7.

Variable	Value
`params.exercise.To_min`	$T_{o,\min}$
`params.exercise.To_max`	$T_{o,\max}$
`params.exercise.eta_min`	$\boldsymbol{\eta}_{\min}$
`params.exercise.eta_max`	$\boldsymbol{\eta}_{\max}$

Table 7: Disturbance bounds given in `generate_params_cc`.

Tasks

24. For the disturbance term $\boldsymbol{w} := B_d \boldsymbol{d}(k)$, find a matrix $H_w \in \mathbb{R}^{2n_x \times n_x}$ and a vector $\boldsymbol{h}_w \in \mathbb{R}^{2n_x}$ such that $P_w = \{\boldsymbol{w} \mid H_w \boldsymbol{w}(k) \leq \boldsymbol{h}_w\}$ is the minimal polytope containing all possible disturbances $\boldsymbol{w}$. Implement the function `generate_params_robust_cc` that takes as an input the `params` struct and adds fields for the polytopic disturbance bound according to Table 8, and then returns the enhanced struct `params_robust`. 2 *pt.*

Field	Value
`constraints.DisturbanceMatrix`	H_w
`constraints.DisturbanceRHS`	h_w

Table 8: Added fields in the `params_robust` struct, the other fields should be the same as in the `params` struct.

25. Sample a sequence W_{sim} of N_{sim} disturbance vectors $\boldsymbol{w}(k)$, i.e.,

$$W_{\text{sim}} := \begin{bmatrix} \boldsymbol{w}(0) & \dots & \boldsymbol{w}(N_{\text{sim}}-1) \end{bmatrix} \in \mathbb{R}^{n_x \times N_{\text{sim}}}.$$

Each disturbance vector $\boldsymbol{w}(k)$, $k = 0, \dots, N_{\text{sim}}$ is sampled from $\{\boldsymbol{w}(k) \in \mathbb{R}^{n_x} \mid H_w \boldsymbol{w}(k) \leq \boldsymbol{h}_w\}$ and w(k) should be generated randomly, i.e. $w(k)$ should generally vary for different k and repeated generation of disturbance sequences. Implement a function `generate_disturbances_cc` that takes `params_robust` as input and outputs W_{sim}. 1 *pt.*

26. Simulate the closed-loop system in the presence of additive disturbances. Adapt the simulation function developed in Task 8 to implement a function `simulate_uncertain`, which takes as inputs $\boldsymbol{x}_0$, `ctrl`, W_{sim}, `params_robust`, and outputs X_{sim}, U_{sim}, and the N_{sim}-dimensional struct `ctrl_info` just as in Task 8, but this time for the uncertain system model (22). 1 *pt.*

In the following, you will implement a tube-based robust MPC controller[5] of the form

$$\min_{Z,V} \; l_f(\boldsymbol{z}_N) + \sum_{i=0}^{N-1} \boldsymbol{z}_i^\top Q \boldsymbol{z}_i + \boldsymbol{v}_i^\top R \boldsymbol{v}_i \tag{24a}$$

$$\text{s.t.} \quad \boldsymbol{x}(k) \in \boldsymbol{z}_0 \oplus \mathcal{E} \tag{24b}$$

$$\boldsymbol{z}_{i+1} = A\boldsymbol{z}_i + B\boldsymbol{v}_i, \qquad i = 0, \dots, N-1 \tag{24c}$$

$$\boldsymbol{z}_i \in \mathcal{X} \ominus \mathcal{E}, \qquad i = 0, \dots, N-1 \tag{24d}$$

$$\boldsymbol{v}_i \in \mathcal{U} \ominus K_{\text{tube}}\mathcal{E}, \qquad i = 0, \dots, N-1 \tag{24e}$$

$$\boldsymbol{z}_N \in \mathcal{X}_N, \tag{24f}$$

[5] Lec. 9, Robust MPC, Tube-MPC Problem Formulation

where $Z := \{z_0, \dots z_N\}$ and $V := \{\boldsymbol{v}_0, \dots \boldsymbol{v}_{N-1}\}$ define a sequence of nominal states and inputs, $\mathcal{E}$ is a robust positively invariant set[6], K_{tube} a stabilizing linear feedback controller for system (22), and $\mathcal{X} = \{\boldsymbol{x} \in \mathbb{R}^{n_x} \mid H_x \boldsymbol{x} \leq \boldsymbol{h}_x\}$ and $\mathcal{U}_z = \{\boldsymbol{u} \in \mathbb{R}^{n_u} \mid H_u \boldsymbol{u} \leq \boldsymbol{h}_u\}$ denote the polytopic state and input constraints. At each time step k, the following control input is applied to the system:

$$\boldsymbol{u}(k) = \kappa_{\text{tube}}(\boldsymbol{x}(k)) = \boldsymbol{v}_0^* + K_{\text{tube}}(\boldsymbol{x}(k) - z_0^*). \tag{25}$$

27. Tube MPC relies on a robust positively invariant (RPI) set $z_k \oplus \mathcal{E}$ and the tube controller K_{tube} to ensure that the system remains inside the set $\mathcal{E}$ for all time steps k when starting from $\boldsymbol{x}(0) \in z(0) \oplus \mathcal{E}$ and applying the tube controller K_{tube}. A polytopic candidate set $\mathcal{E}$, defined as in Equation 26, and the corresponding candidate tube controller K_{tube} are provided in the `params` struct, with naming according to Table 9.

$$\mathcal{E} := \{\boldsymbol{x} \in \mathbb{R}^{n_x} \mid H_{\text{tube}} \boldsymbol{x} \leq \boldsymbol{h}_{\text{tube}}\}, \tag{26}$$

Implement the function `check_RPI` that takes as input the `params_robust` struct, the

Field	Value
`exercise.H_tube`	H_{tube}
`exercise.h_tube`	$\boldsymbol{h}_{\text{tube}}$
`exercise.K_tube`	K_{tube}

Table 9: Candidate polytopic tube given in the `params` struct.

candidate tube given by H_{tube}, h_{tube}, the disturbance set given by H_w, h_w, the tube controller K_{tube}, and verifies whether the cadidate set (26) is an RPI set for system (22) under the tube controller K_{tube}. The function should return a boolean value `is_rpi` equal to `true` if the set is RPI and equal to `false` otherwise.
Hint: You can use the method `Polyhedron.contains()` *to check for set containment. Refer to the MPT3 github linked here for more information.* 1 *pt.*

28. *[Self-study] In order to obtain the largest feasible set for the Tube MPC, the tube $\mathcal{E}$ is ideally chosen as the minimal RPI set. How could you obtain the minimal RPI set? If you implement the respective algorithm, try to run the first few iterations. Why could it be computationally hard to calculate the minRPI set in this case? Can you come up with an RPI set that has a lower volume than the provided set?*
Hint 1: It can be computationally challenging to compute the minimal RPI set.
Hint 2: You can check the volume of a polyhedron using its method `.volume()`.

29. Obtain the tightened constraints as defined in the MPC problem (24). Let H_x, $\boldsymbol{h}_x$, H_u, and $\boldsymbol{h}_u$ define the polytopic constraints for the system (22) as given in `params_robust`. Implement a function `compute_tightening`, which takes `params_robust` as input and adapts the parameters H_x, $\boldsymbol{h}_x$, H_u, and $\boldsymbol{h}_u$ to represent the tightened constraints in (24d) and (24e). The output is then given by the modified struct `params_robust_tube`. 3 *pt.*

30. Implement the tube-based robust model predictive controller (24) in the class `MPC_TUBE`. Design the terminal cost $l_f(\boldsymbol{x})$ as the LQR infinite-horizon cost (13) of the nominal system, i.e., system (22) without disturbances $w(k)$. During initialization of the class, a YALMIP solver object representing the optimization problem (24) needs to be created based on the inputs Q, R, N, the polytopic terminal set described by H_N and $\boldsymbol{h}_N$, the polytopic tube $\mathcal{E}$ described by H_{tube}, $\boldsymbol{h}_{\text{tube}}$, the tube controller K_{tube}, as well as the system model with tightened constraints in the struct `params_robust_tube`. The `eval` method should solve the optimization problem and obtain the control input according to (25). 4 *pt.*

[6] Lec. 8/9, Robust MPC, Robust Invariant Set

31. Let $Q = \text{diag}(q^*)$, $R = diag(r^*)$, $N = N_{sim}$. Use the provided tube controller K_{tube} and tube $\mathcal{E}$ provided in the `params` struct (according to Equation (26) and Table 9) and check that $\mathcal{E}$ is RPI under the tube controller K_{tube} using your function `check_RPI` as described in Task 27. Obtain tightened constraints as outlined in Task 29. Using the function `lqr_maxPI` with an appropriate choice of inputs, design the terminal set $\mathcal{X}_N$ such that problem (24) is recursively feasible and system (22) under controller (25) is robustly stable. Write the design of all ingredients and the setup of the resulting `MPC_TUBE` controller in a script called called `MPC_TUBE_script_cc.m`. Provide the script and save the following variables in the MAT-file `MPC_TUBE_params_cc.mat`: `K_tube` $:= K_{\text{tube}}$, `H_tube` $:= H_{\text{tube}}$, `h_tube` $:= \boldsymbol{h}_{\text{tube}}$, `H_N` $:= H_N$, `h_N` $:= \boldsymbol{h}_N$, and `params_robust_tube`. Strictly use the naming convention specified in Table 10. 2 *pt.*

32. *[Simulation]: Sample different disturbance sequences W_{sim} and simulate the MPC-TUBE controller obtained in Task 31 using your function `simulate_uncertain` for initial condition $\boldsymbol{x}_0^A$. Compare your MPC-TUBE controller against applying the MPC-TS controller implemented in Task 18 based on the system defined in `params_robust` with $N = N_{sim}$. Plot the closed-loop trajectories using the function `plot_trajectory`. What can you observe? What can you observe if you apply the maximum disturbance at each time step, i.e., $w(k) = \begin{bmatrix} w_{1,\max} & w_{2,\max} & w_{3,\max} \end{bmatrix}^\top$ for all time steps k?*

Name	Variable
`K_tube`	K_{tube}
`H_tube`	H_{tube}
`h_tube`	$\boldsymbol{h}_{\text{tube}}$
`H_N`	H_N
`h_N`	$\boldsymbol{h}_N$
`params_robust_tube`	Output of `compute_tightening`

Table 10: Variable naming convention for `mpc_tube_params_cc.mat`